

1-5 什么是多道程序设计技术？试述多道程序运行的特征？

多道程序设计技术是指在计算机主存中同时存放几道相互独立的程序。这些程序在执行系统控制之下，相互穿插地运行。当某道程序因某种原因不能继续运行下去时(如等待外部设备传输数据)，管理程序便将另一道程序投入运行。这样可以最大限度地利用 CPU 时间，避免因一个程序等待 I/O 而导致的 CPU 空闲。

多道运行的特征

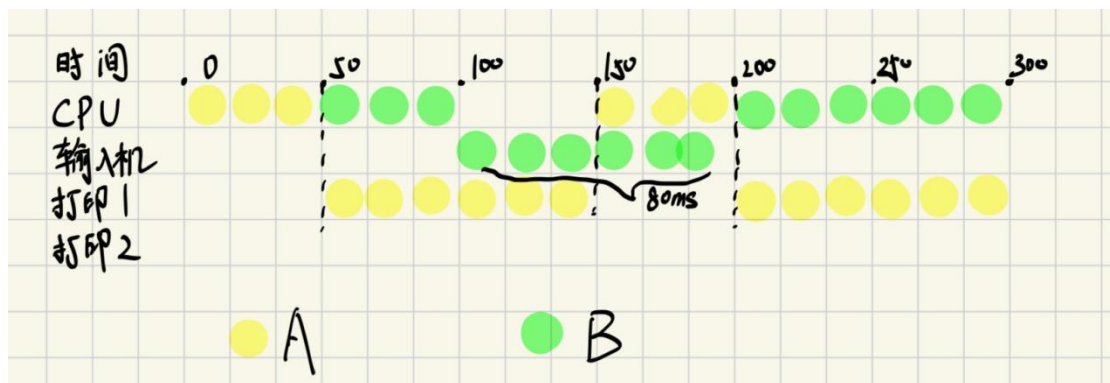
多道 （内存中存放多个相互独立的程序）

宏观上并行 （相互穿插运行）

微观上串行 （CPU 上永远只能执行单个程序）

1-9 设一计算机系统有输入机一台、打印机两台，现有 A、B 两道程序同时投入运行，且程序 A 先运行，程序 B 后运行。程序 A 的运行轨迹为：计算 50ms，打印信息 100ms，再计算 50ms，打印信息 100ms，结束。程序 B 运行的轨迹为：计算 50ms，输入数据 80ms，再计算 100ms，结束。回答如下问题。

(1) 用图画出这两道程序并发执行时的工作情况。



(2) 说明在两道程序运行时，CPU 有无空闲等待？若有，在哪段时间内等？为什么会空闲等待？

CPU 有空闲等待，在 100-150ms 这段时间内空闲等待，因为当 B 计算 50ms 后等待数据输入，而 A 尚在打印时 CPU 空闲，空闲 50ms，直到 A 再次计算数据

(3) 程序 A、B 运行时有无等待现象?在什么时候会发生等待现象?

程序 A: 无明显的 CPU 等待现象, 但其打印操作属于 I/O 等待 (50-150ms 和 200-300ms), 这是正常的 I/O 阻塞等待打印机资源完成。

程序 B: **有等待现象**, 在 180-200ms 发生等待 (**持续 20ms**)。这是因为 B 的输入 I/O 结束返回就绪时, CPU 正被 A 的计算占用, B 必须等待 A 计算完成并切换到 I/O 后才能获得 CPU。

2-8 IRQ 和异常有什么不同

IRQ (中断请求) 与异常的主要区别在于产生原因、触发时机和处理方式不同。

异常是由于当前程序执行过程中产生的**同步事件**, 即由 CPU 在执行某条指令时检测到的错误或特殊情况引发, 如除零错误、非法内存访问、缺页错误等; 它与程序运行密切相关, 发生的时间和指令执行是同步的, 处理后通常返回出错指令重新执行或终止程序。

而 **IRQ** 是由外部设备产生的**异步事件**, 与当前程序的执行无关, 例如键盘敲击、定时器到期、磁盘 I/O 完成等; CPU 在执行完当前指令后响应 IRQ, 转入中断处理程序, 处理完成后返回下一条指令继续执行。

总结来说, 异常反映程序内部问题或特殊状态, 是同步的; IRQ 反映外部设备请求, 是异步的。

3-4 静态链接和动态链接有什么不同?

静态链接是指在程序编译阶段, 将所有需要的外部函数或库文件的代码直接拷贝并链接到目标文件中, 生成一个完整独立的可执行文件。这样, 程序在运行时不再依赖外部库, 即使系统中没有对应的库文件, 也能独立运行。

其**优点**是: 程序运行稳定、加载速度快、无需依赖外部环境;

但**缺点**是: 如果多个程序都使用同一个库函数, 每个程序的可执行文件中都会各自包含该函数的副本, 造成磁盘和内存的浪费, 而且当库文件更新时, 所有使用该库的程序都需要重新链接生成。

动态链接则不同, 它在编译时不把库函数的代码直接复制到目标文件中, 而是仅在程序中记录下所需外部函数的名称和引用位置。程序运行时, 操作系统会根据这些记录, 在执行时动态地加载相应的库文件 (如 Windows 的 DLL 或 Linux

的.so)，并把所需函数链接到程序中。

其**优点**是：节省磁盘和内存空间，多个程序可以共享同一个库文件，更新库文件时无需重新编译应用程序；

但**缺点**是：程序运行时必须依赖外部库文件的存在与正确版本，若系统找不到对应的库或版本不兼容，程序将无法正常运行。

3-8 什么是系统调用？对操作系统的服务请求与一般的子程序调用有什么区别？

系统调用（System Call）是用户程序向操作系统请求服务的接口。用户程序在运行过程中，如果需要执行一些只有操作系统才能完成的特权操作（如文件读写、进程创建、内存分配、设备访问等），就必须通过系统调用进入内核，由操作系统代为完成。这是用户程序与操作系统之间进行交互的**主要接口**。

系统调用与一般的子程序调用有以下区别：

运行级别不同：系统调用运行在**内核态**，涉及用户态到内核态的模式切换（通过软中断或特定指令，如 `int` 或 `syscall`）。操作系统内核直接操作硬件或核心数据结构。子程序调用运行在**用户态**，仅在用户程序的地址空间内执行，调用程序中的函数或库函数。

权限不同：系统调用需要特权操作，访问受限资源（如硬件、文件系统、内存管理），由操作系统内核以高权限执行。子程序调用无特权要求，仅操作用户程序的数据和逻辑，权限受限。

执行机制不同：子程序调用在用户程序中通过普通的 `call` 指令实现，而系统调用通常通过特殊指令触发“**陷入**”机制，由 CPU 自动切换到内核态执行。